

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – PSEUDOCODE WRITING TASK

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 1 – Pseudocode Writing Task
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	G. Nithya Sree	Reg. No.:	192424103
Section / Batch:	3 RD Year AIDS	Date:	28/08/2026

Code of Conduct

I G. Nithya Sree (192424103) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G. Nithya Sree

Instructions

- This test contains 5 Pseudocode Writing Task questions. Total: 100 Marks.
- When a student answer using pseudocode writing task should evaluate based on the ability to design clear, logical, and efficient pseudocode solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

GOLAMARY NITHYA SREE
192424103RB

QUESTIONS

- Q1** Minimum Spanning Tree - Kruskal's Algorithm 20 marks
- Q2** Graph - Topological Sort 20 marks
- Q3** Shortest Path Algorithm - MST & Dijkstra's Algorithm 20 marks
- Q4** Shortest Path Algorithm - Dijkstra's Algorithm 20 marks
- Q5** Minimum Spanning Tree - Prim's or Kruskal's Algorithm 20 marks

Q1 Minimum Spanning Tree - Kruskal's Algorithm 20 marks

Write pseudocode for constructing an MST using Kruskal's Algorithm. A city planning department needs to connect multiple zones with underground fiber-optic cables. Each zone is represented as a vertex, and possible cable connections between zones are represented as edges with installation costs (weights).

Constraints:

- All zones must be connected
- Total installation cost must be minimized
- No redundant connections (no cycles) allowed
- The network must remain fully connected

Your Answer

```

Algorithm Kruskal_City_MST(Zones, CableConnections):
  // Input:
  // Zones = set of all city zones (vertices)
  // CableConnections = set of all possible cable links with installation costs (edges)

  // Step 1: Initialize an empty set to store the final optimal network
      
```

Save Answer
 Next →

← → ↺

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=517

🔒 ☆ 📄 👤

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

GOLAMARY NITHYA SREE
192424103RB

QUESTIONS

Q1
Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2
Graph - Topological Sort
20 marks

Q3
Shortest Path Algorithm - MST
& Dijkstra's Algorithm
20 marks

Q4
Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks

Q5
Minimum Spanning Tree -
Prim's or Kruskal's Algorithm
20 marks

Q2 Graph - Topological Sort 20 marks

A DAG represents task dependencies. Write pseudocode for Topological Sort using DFS with cycle detection.

Constraints:

- Detect cycle before sorting
- Use DFS approach

Your Answer

```
Algorithm TopologicalSort_DFS(Graph):  
    // Graph contains a set of Vertices and Edges representing task dependencies  
  
    // Initialize data structures  
    Visited = Empty Set    // Tracks nodes that have been completely processed  
    RecursionStack = Empty Set    // Tracks nodes in the current DFS path (for cycle detection)
```

Save Answer

Next →

← → ↺

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=517

🔒 ☆ 📄 👤

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

GOLAMARY NITHYA SREE
192424103RB

QUESTIONS

Q1
Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2
Graph - Topological Sort
20 marks

Q3
Shortest Path Algorithm - MST
& Dijkstra's Algorithm
20 marks

Q4
Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks

Q5
Minimum Spanning Tree -
Prim's or Kruskal's Algorithm
20 marks

Q3 Shortest Path Algorithm - MST & Dijkstra's Algorithm 20 marks

Write pseudocode integrating MST + Dijkstra for Smart transportation system.

Constraints:

- Use MST for infrastructure
- Use shortest path for routing

Your Answer

```
Algorithm SmartTransportationSystem(CityGraph, SourceZone, DestinationZone):  
    // Phase 1: Infrastructure Planning (Addresses Constraint a: Use MST for infrastructure)  
    // Find the minimum cost required to build transport links connecting all city zones.  
    InfrastructureNetwork = Build_MST_Infrastructure(CityGraph.Vertices, CityGraph.Edges)  
  
    // Phase 2: Route Navigation (Addresses Constraint b: Use shortest path for routing)
```

Save Answer

Next →

← → ↻

Not secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=517

🔑 ☆ 🏠 👤

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

GOLAMARY NITHYA SREE
192424103RB

QUESTIONS

Q1

Minimum Spanning Tree -
Kruskal's Algorithm

20 marks

Q2

Graph - Topological Sort

20 marks

Q3

Shortest Path Algorithm - MST
& Dijkstra's Algorithm

20 marks

Q4

Shortest Path Algorithm -
Dijkstra's Algorithm

20 marks

Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm

20 marks

Q4 Shortest Path Algorithm - Dijkstra's Algorithm

20 marks

Shortest Path with "Toll-Free" Voucher: You have a voucher that allows you to traverse one edge for free (weight = 0). Write pseudocode to find the shortest path from S to D utilizing this voucher optimally.

Your Answer

```
Algorithm ShortestPath_With_Voucher(Graph, S, D):  
    // We maintain two distances for each vertex:  
    // Distance[v][0] = Shortest distance to v WITHOUT using the voucher  
    // Distance[v][1] = Shortest distance to v HAVING USED the voucher  
    FOR each vertex v in Graph.Vertices DO:  
        Distance[v][0] = INFINITY
```

Save Answer

Next →

← → ↻

Not secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=517

🔑 ☆ 🏠 👤

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

GOLAMARY NITHYA SREE
192424103RB

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

Q1

Minimum Spanning Tree -
Kruskal's Algorithm

20 marks

Q2

Graph - Topological Sort

20 marks

Q3

Shortest Path Algorithm - MST
& Dijkstra's Algorithm

20 marks

Q4

Shortest Path Algorithm -
Dijkstra's Algorithm

20 marks

Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm

20 marks

Q5 Minimum Spanning Tree - Prim's or Kruskal's Algorithm

20 marks

Second Best Spanning Tree: Design pseudocode to find the "Second Best" MST. (20 Marks)
Hint: Find the MST first, then for every edge in the MST, try removing it and finding the next best edge to replace it.

Your Answer

```
Algorithm SecondBest_MST(Vertices, Edges):  
    // Step 1: Find the original Minimum Spanning Tree (using Kruskal's or Prim's)  
    OriginalMST = Find_MST(Vertices, Edges)  
  
    // Initialize variables to track the second best tree  
    SecondBestCost = INFINITY
```

Save Answer